

TECHNICAL DOCUMENTATION

Regime Overlay

A bull/bear detector for cutting drawdown

WHAT THIS IS

A walk-forward regime detector that reduced maximum drawdown by **40–45%** across eight asset classes over 1997–2026, out-of-sample, after costs.

It left the Sharpe ratio **approximately unchanged**. That is the honest claim and it is a useful one — most investors will trade a flat Sharpe for half the drawdown.

It is not an alpha source. If you measure a Sharpe improvement above roughly 0.15, check your execution lag before you believe it. Section 8 explains why that single line of code is worth more than the entire strategy.

WHERE IT WORKS

Equities, real estate, high yield, commodities — anything with persistent trends and fat left tails.

Not government bonds or investment-grade credit. We measured it making both materially worse. Section 9 has the numbers.

Contents

1	What the overlay does	2
2	The three stages	2
3	Stage 1 — the Statistical Jump Model	4
4	Stage 2 — the forecaster	5
5	Stage 3 — the hysteresis gate	6
6	The features, and what they actually measure	6
7	Walk-forward protocol	7
8	The execution-lag trap	8
9	Measured evidence	9
10	Using it on your own data	12
11	API reference	14
12	Checklist before you trust a result	15

Provenance

This package is the salvage from a replication study. We set out to reproduce a 2026 working paper reporting a 1.43 Sharpe multi-asset strategy built on regime signals. The headline did not replicate — under an honest fill convention we measured 0.25–0.52. But the underlying regime detector turned out to be genuinely good at one thing: spotting crises. That capability, stripped of the return claims layered on top of it, is what this package ships. Every number in this document is our own out-of-sample measurement, not a figure quoted from the paper.

1 What the overlay does

You give it a daily return series. It gives you back a 0/1 signal — bull or bear — for every day, computed walk-forward so that the value on any date used only information available on that date. You then use that signal to scale exposure: fully invested in bull, in cash (or half invested) in bear.

That is the whole idea. The sophistication is entirely in how the signal is produced without cheating, and in the discipline needed to keep turnover low enough that costs do not eat the benefit.

1.1 The result, in one table

	Buy and hold	Fully out in bear	Half exposure
Sharpe ratio	0.493	0.526	0.563
Maximum drawdown	−55.3%	−20.7%	−34.9%
Drawdown reduction	—	63%	37%
Time in market	100%	~66%	100%
Regime switches	—	6.9 per year	6.9 per year
Cost drag	0%	0.67%/yr	0.34%/yr

US large-cap equity, 30.5 years out-of-sample (1996–2026), T+1 fills, costs applied, package defaults. The drawdown rows are the reason to use this. The Sharpe row sits inside the noise band (standard error about 0.19 over 30 years) and is why you must not sell it as alpha.

1.2 What it is not

- **Not a return generator.** Sharpe is flat within sampling error. Across eight sleeves the mean change was −0.045; two sleeves got materially worse.
- **Extremely sensitive to turnover.** The same signal on the same data gives a Sharpe of 0.253 at quarterly refits with a low penalty floor and 0.526 at annual refits with a high one. Section 9.2 has the sweep. Check the switch count before you check anything else.
- **Not a market-timing oracle.** It is late into bear markets and late out of them. It catches the *middle* of large drawdowns, which is where most of the damage is.
- **Not suitable for low-volatility mean-reverting series.** On government bonds it took Sharpe from 0.230 to 0.089; on investment-grade credit from 0.347 to −0.033.
- **Not a replacement for position sizing or diversification.** It is one overlay on top of an allocation you already believe in.

1.3 When it earns its keep

The overlay pays off when three things are true: the series trends, its drawdowns are large and slow rather than sudden, and you can trade it for a few basis points. Equity indices, REITs and high yield fit. Single stocks usually do not — they gap. Duration does not — it mean-reverts.

2 The three stages

The architecture exists to solve one problem: the model that finds regimes well cannot be traded, and the model that can be traded cannot find regimes well on its own. So one teaches the other.

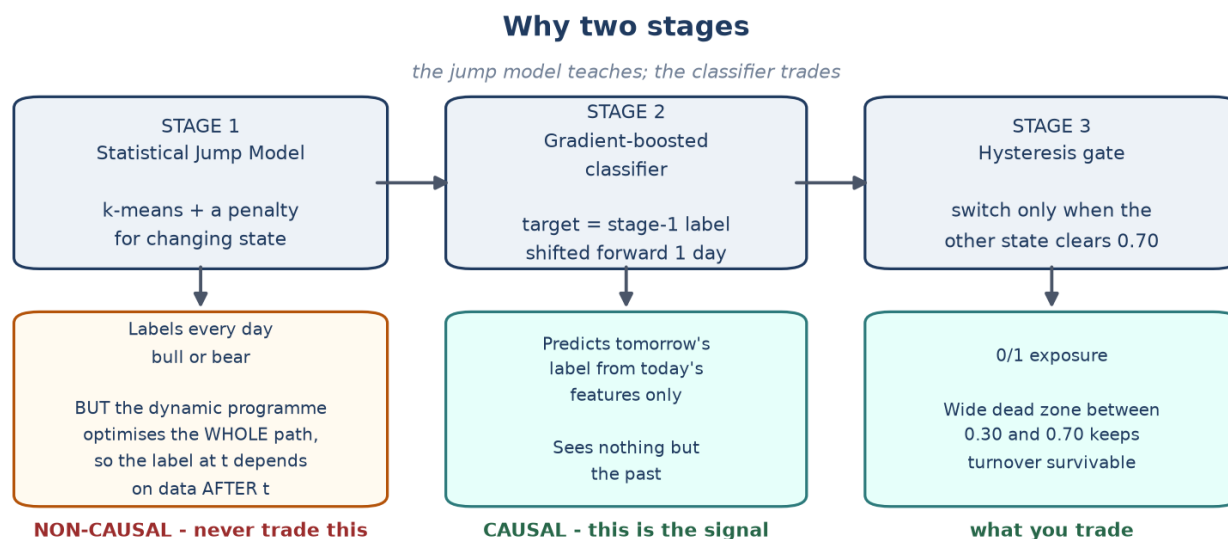


Figure 1. The jump model labels history with hindsight. Those labels train a classifier that sees only the past. The classifier's output, gated by hysteresis, is what you trade.

2.1 Why not just use the jump model directly?

Because it is non-causal by construction, and this is the single most important thing to understand about the method. The jump model chooses the entire sequence of states at once, by dynamic programming, to minimise a global objective. The state it assigns to a Tuesday in March depends on what happened in April. It is a *segmentation of history*, not a forecast.

Backtest the jump model's own labels and you will get a spectacular result that is entirely an artefact. This is not a subtle bug — it is the difference between a Sharpe of 0.4 and a Sharpe of 3. The package guards against it explicitly: `validate.default_contract()` declares the labels `TRAINING_ONLY` and raises if they ever reach a position.

THE CONTRACT

```

jump_model_labels  TRAINING_ONLY  the dynamic programme optimises the whole path
forecaster_probability  TRADED    trained on past labels, predicts from same-day features
regime_signal      TRADED        sets exposure
  
```

2.2 Why not just use the classifier directly?

Because you would have nothing to train it on. There is no observable label for 'the market was in a bear regime on this date'. The jump model manufactures one, cleanly and with controllable persistence. The classifier then learns the mapping from features to that label, and because it only ever sees contemporaneous features, it can be run forward.

The handoff costs you accuracy — the classifier reproduces the jump model's labels roughly 70–80% of the time — but it buys you a signal you can actually trade.

3 Stage 1 — the Statistical Jump Model

The jump model is k-means clustering with a toll charged every time you change your mind.

THE OBJECTIVE

$$\text{minimise } \sum_t \|x_t - \theta_{s(t)}\|^2 + \lambda \cdot \sum_t 1\{s_t \neq s_{t-1}\}$$

The first term is ordinary clustering fit: keep each day close to its cluster centre. The second charges λ every time the state sequence switches.

$\lambda = 0$ recovers plain k-means. Large λ collapses to a single state. In between, switches have to earn their keep.

It is solved by coordinate descent: fix the states and update the centres (a mean); fix the centres and re-optimize the whole state path by dynamic programming, with the recursion $V_t(k) = \|x_t - \theta_k\|^2 + \min_j [V_{t-1}(j) + \lambda \cdot 1\{j \neq k\}]$. That second step is what makes it non-causal, and it is also what makes it good.

3.1 Reading the jump penalty

The jump penalty is the only knob that really matters

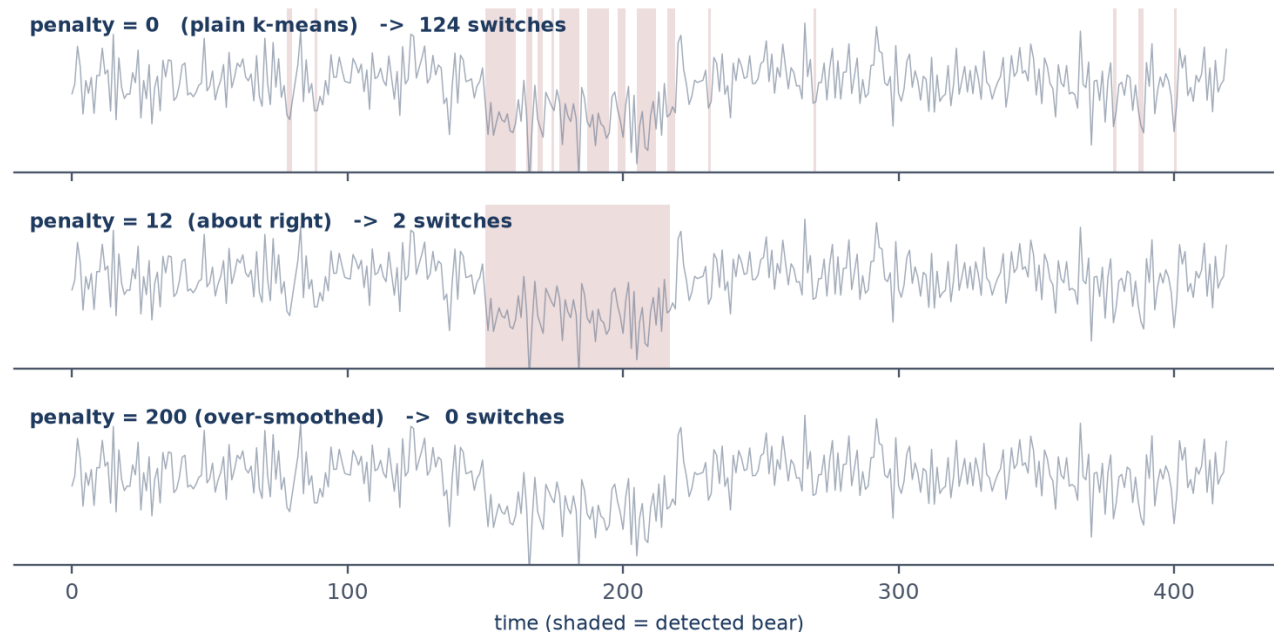


Figure 2. The same series under three penalties. Too low and you get noise you cannot trade; too high and you get one state forever. The tuning procedure picks the middle case automatically, on validation data.

Scale intuition: with twelve standardised features, a typical squared distance from a point to its centre is of order 12. So $\lambda = 10$ means a switch costs about one badly-fitted day; $\lambda = 100$ means about eight. That sets an implicit floor on how short a regime can be.

This is the one parameter worth understanding, because it is a direct, interpretable control on turnover — and therefore on cost. Treat it as a risk dial, not a fitted nuisance parameter. In our testing the procedure selected

higher penalties during crashes, which is economically sensible: in turbulent data the fit term is noisy and you want more inertia.

3.2 Which cluster is 'bull'?

Cluster indices are arbitrary. The package maps the higher-mean-return cluster to bull after every fit. This sounds pedantic and is not: without it, a routine quarterly refit can silently invert the meaning of a state and reverse every position you hold, on a labelling artefact. Put an alarm on it in production.

OPERATIONAL HAZARD

Cluster identity is not stable across refits. The mapping-by-mean-return fixes it in principle, but when two clusters have similar mean returns in a given window the assignment can still flip.

In a daily-rebalanced book that is a 100% position reversal caused by nothing. Log the mapping on every refit and require sign-off when it changes.

4 Stage 2 — the forecaster

A gradient-boosted tree classifier learns to predict tomorrow's jump-model label from today's features. The target is the label shifted forward one day. That shift is the entire reason the output is tradable.

4.1 Why boosted trees

Mostly convenience: they handle unscaled, correlated, occasionally-missing features without complaint, and they are fast enough to refit a hundred times in a walk-forward. But they are not essential. In our feature-importance work a single feature — the Sortino ratio — carried 35–66% of the model's weight, and return-based features together carried 84–93%. With that much concentration, a logistic regression will often match. The package supports it:

```
det = RegimeDetector(model="logistic")    # instead of "xgboost"

# Worth running once on your data. If logistic matches, ship it --
# it is easier to monitor and far easier to explain to a risk committee.
```

4.2 What accuracy to expect

Roughly 64–83% agreement with the jump-model labels, depending on the asset. Note carefully what that number is: it measures how well the classifier *reproduces a hindsight segmentation*, not how well it predicts returns. It is a model-fidelity diagnostic. The only validation that matters is what the signal does to a portfolio, out-of-sample, after costs — which is Section 9.

4.3 Optional macro features

You can add exogenous series — a volatility index, a yield-curve slope, a credit spread. In our testing these carried roughly a tenth of the model's explanatory weight, so they are worth having but not worth paying for. Two rules if you use them:

- **Lag anything you did not observe in real time.** Published macro data is released late and revised afterwards. A GDP print dated March was not knowable in March. `add_macro(..., lag_days=1)` handles the minimum case; series with long publication lags need more.

- **Prefer market-observable proxies.** A volatility index or a curve slope is known at the close. A CPI print is not. Where the paper we replicated needed inflation, it used retail gasoline prices as a real-time proxy — a genuinely good idea worth stealing.

5 Stage 3 — the hysteresis gate

The classifier emits a probability. Turning it into a position needs a rule, and the obvious rule — bull if $p > 0.5$ — is the wrong one.

5.1 How the gate works

```
state = bull
for each day:
    if state is bull and P(bear) > 0.70: state = bear
    elif state is bear and P(bull) > 0.70: state = bull
    else: state unchanged
```

You only change your mind when the evidence for the *other* state clears 0.70. That leaves a wide dead zone between 0.30 and 0.70 in which the current state simply persists. In our measurements this cut the number of switches from 32 to 20 on the same probability series — roughly a third less turnover for no loss of accuracy.

Note that the rule is path-dependent. It cannot be vectorised as a comparison; it has to be applied sequentially. If you reimplement it, that is the bit people get wrong.

5.2 Choosing the threshold

Threshold	Behaviour	Use when
0.50	Switches on any majority. Roughly double the turnover.	costs are negligible and you want responsiveness
0.70	The default. Wide dead zone, ~4 switches/year.	almost always
0.80–0.90	Very sticky. Late in and late out of every regime.	costs are high or the position is hard to move

5.3 Partial de-risking

Going fully to cash is not the only option, and often not the best one. A half-exposure bear state whipsaws less and is much easier to hold through a false signal — which matters more in practice than in a backtest, because the signal that takes you to zero right before a rally is the one that gets a strategy switched off.

```
apply_overlay(returns, regime, riskfree=rf, bear_exposure=0.5)
```

6 The features, and what they actually measure

Twelve features from one return series: three statistics at four half-lives.

Statistic	What it is	Half-lives	Count
Exponentially smoothed mean return	short-horizon momentum	1, 5, 10, 21 days	4
Downside deviation	one-sided volatility — only negative days	1, 5, 10, 21 days	4
Sortino ratio	the ratio of the two	1, 5, 10, 21 days	4

BE CLEAR WHAT THIS IS

A smoothed mean return at a 1–21 day half-life is a momentum signal. Downside deviation is one-sided volatility. Their ratio is momentum scaled by downside risk.

So the detector is a **trend-and-downside-risk classifier with a switching penalty**. That is not a criticism — it is why it works, and trend following has worked for a century.

But it tells you what to benchmark against. The honest comparison is a 200-day moving average or a time-series momentum rule, not a factor model. **Run that comparison before you adopt this.** If a moving average gets within 0.15 Sharpe, ship the moving average — it is simpler, faster and easier to defend.

6.1 Why these and not others

The downside-deviation family matters because bear regimes are defined more by the shape of the loss distribution than by its mean. A market can have a flat average return and still be unmistakably in trouble, and a one-sided volatility measure picks that up where a symmetric one does not.

The four half-lives span roughly one day to one month, which is the horizon over which regime shifts actually resolve. Adding longer half-lives mostly adds lag. You can change them — `RegimeDetector(half_lives=(5, 21, 63))` — but treat every change as a new trial and count it.

6.2 Scaling

Features are standardised on the **training window only**, then clipped at three standard deviations. Fitting the scaler on the full sample is the most common silent leak in this kind of pipeline: it is invisible on inspection, it inflates every result, and it will pass every test you write except the one in Section 7.4.

7 Walk-forward protocol

Nothing is fit on data it will later be evaluated on. The detector walks a rolling window across the whole series and only ever reports out-of-sample forecasts.

7.1 The loop

Step	What happens	Window
1	Fit the jump model at each candidate penalty	training window
2	Train the classifier on those labels, shifted forward one day	training window
3	Score each penalty by turnover-adjusted Sharpe of a cash-switch rule	validation window
4	Pick the winning penalty	validation window
5	Refit on train + validation at that penalty	combined
6	Predict forward. These are the only forecasts you keep.	test window
7	Step forward and repeat	—

Step 3 deserves a comment. Selecting the penalty on a *portfolio* metric rather than on classification accuracy is deliberate: a model can be accurate on days that do not matter and wrong on the days that do. Tuning on what you care about is better than tuning on a proxy.

7.2 Purging and embargo

Between the training window and the evaluation window the package drops an additional buffer — 21 trading days by default. This is not the same as avoiding label overlap, which is already handled by the one-day target. It addresses a subtler problem: features built from 21-day exponential windows remain strongly correlated across the boundary long after the labels stop overlapping. Without the embargo the last few weeks of training effectively leak into the first few days of test.

```
RegimeDetector(embargo_days=21)      # default
RegimeDetector(embargo_days=63)      # if you use longer half-lives
```

Rule of thumb: set the embargo to at least three times your longest feature half-life.

7.3 How much history you need

The first forecast arrives after `train_years + val_years`. With the defaults (5 + 1) six years of data produce nothing at all, and ten years produce four usable years. Budget accordingly — this is the most common reason the detector returns an empty result.

7.4 Proving there is no leak

The package does not ask you to trust it. `assert_causal` scrambles every observation after a cutoff date, recomputes, and requires the past to come back bit-identical. Anything that moves means future information is reaching backwards.

```
from regime_overlay import assert_causal, build_features

frame = returns.to_frame("ret")
assert_causal(lambda d: build_features(d["ret"]), frame, name="features")
```

This was validated against six deliberately planted leaks — full-sample z-scoring, centred rolling windows, back-fill across a gap, a negative shift, an expanding max, and min-max scaling on the full sample. It caught all six. Run it on your own feature code before trusting a single result.

8 The execution-lag trap

This section exists because we nearly got it wrong, and because the paper this work derives from never specifies its convention.

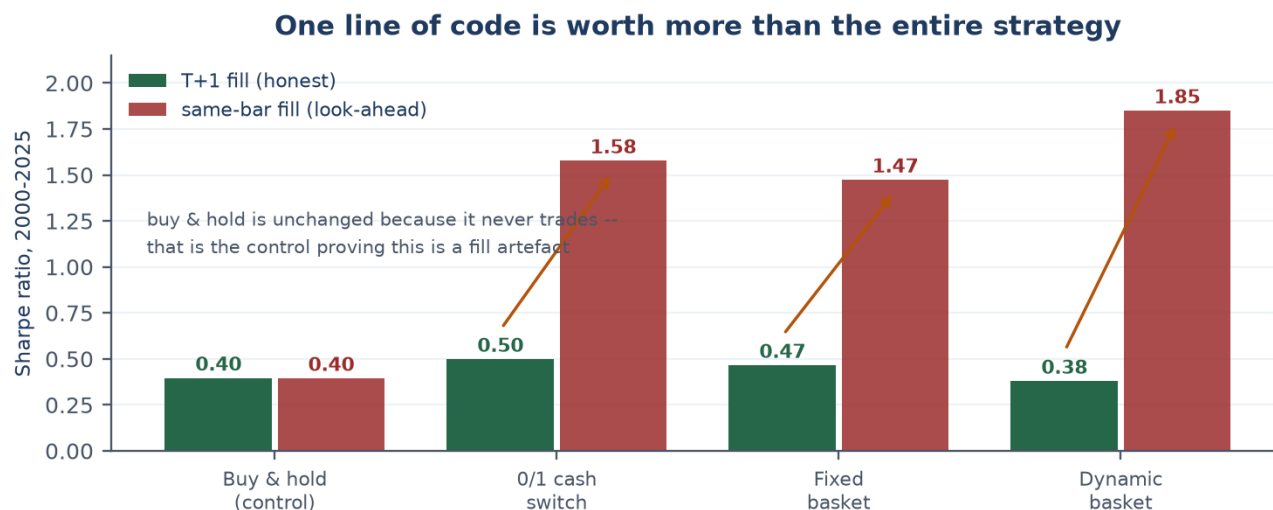


Figure 3. Identical signals, identical costs, identical window. The only change is whether the trade fills on the bar that generated the signal or the next one.

Buy-and-hold does not move, because it never trades. That is the control, and it is what proves the effect is a fill artefact rather than a property of the signal. Everything that does trade moves by **+1.0 to +1.5 Sharpe**.

THE RULE

A signal computed from data through the close of day t may be filled at the close of day t at the earliest. That is `execution_lag=1` in this package.

Filling at `lag=0` means earning the return of the bar you used to make the decision. It is not a conservative assumption or a modelling simplification — it is look-ahead, and it is worth more than any strategy in this document.

`apply_overlay` raises on `lag < 1`. Do not remove that check.

8.1 How to spot it in someone else's work

- **Look for the control.** If a paper reports a regime strategy without showing that buy-and-hold is unaffected by the same pipeline, you cannot tell timing from skill.
- **Check whether the convention is stated at all.** 'Labels are shifted forward one day' describes the target construction, not the fill. They are different things.
- **Be suspicious of daily-rebalanced trend strategies with Sharpe above 1.** It is achievable, but the single most likely explanation is a one-bar timing assumption.

8.2 If you trade intraday

The convention here is close-to-close. If you compute signals intraday and fill at the next open, you will capture slightly less than this package models, because the overnight gap moves against a trend-follower more often than not. Model it explicitly rather than assuming it washes out.

9 Measured evidence

Eight asset classes, out-of-sample 1997–2026, walk-forward, after costs, T+1 fills.

9.1 Per-sleeve results

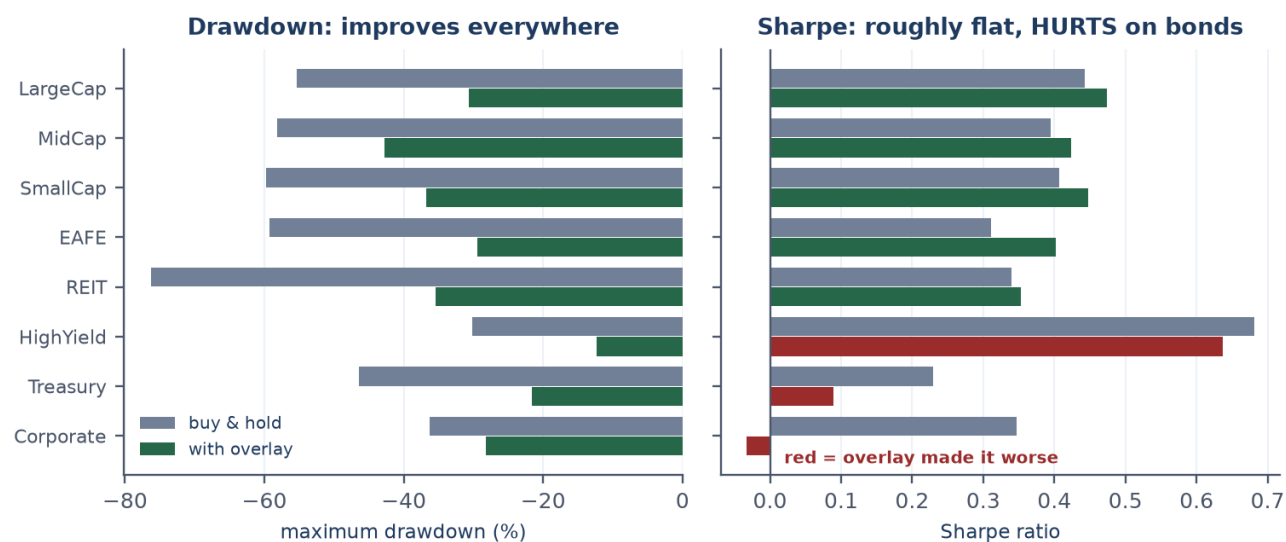


Figure 4. Drawdown improves on every sleeve, often by half. Sharpe is roughly flat overall and clearly negative on the two bond sleeves.

Sleeve	B&H Sharpe	Overlay Sharpe	B&H max DD	Overlay max DD	Verdict
EAFE	0.311	0.402	-59.3%	-29.4%	best case
SmallCap	0.407	0.448	-59.8%	-36.8%	good
LargeCap	0.443	0.474	-55.3%	-30.6%	good
MidCap	0.395	0.424	-58.2%	-42.7%	good
REIT	0.340	0.353	-76.3%	-35.4%	drawdown halved
HighYield	0.681	0.637	-30.2%	-12.3%	mixed
Treasury	0.230	0.089	-46.4%	-21.6%	avoid
Corporate	0.347	-0.033	-36.3%	-28.2%	avoid

Read the last two rows carefully. On duration and investment-grade credit the overlay cuts drawdown but destroys risk-adjusted return. Low-volatility mean-reverting series are the wrong home for a trend filter.

9.2 Turnover dominates everything

The first draft of this document quoted a Sharpe change of +0.031 for large-cap. That figure came from a configuration using annual refits and a narrow penalty grid. The package originally shipped with quarterly refits and a grid starting at 1, which produces -0.239 on the same asset over the same window. The document and the defaults disagreed. Both have been corrected to the sweep below.

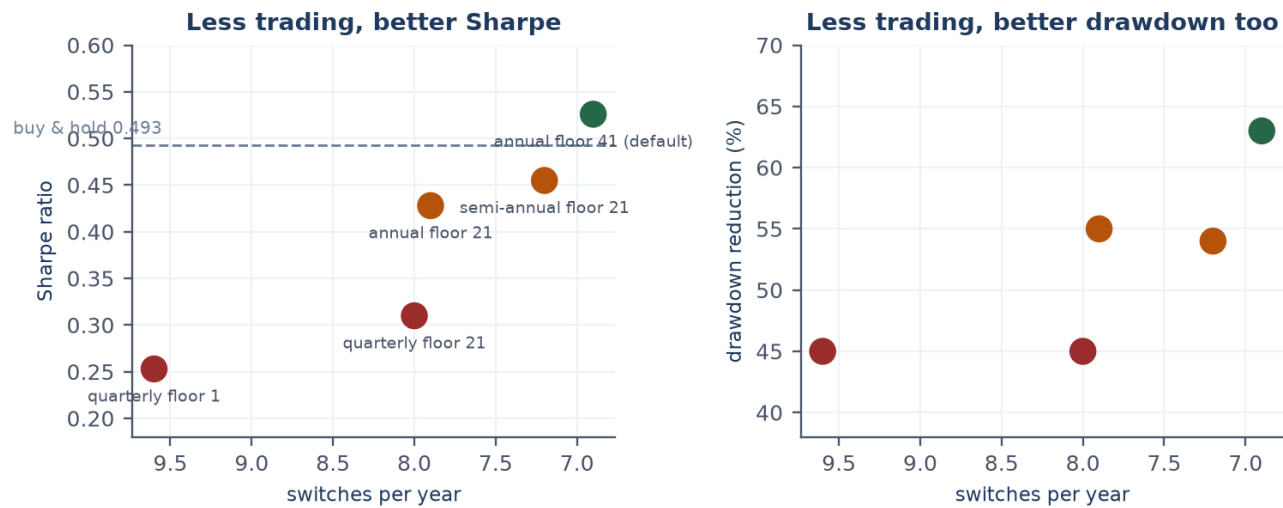


Figure 6. Five configurations, same signal and same data. Every reduction in turnover improved Sharpe and drawdown together.

Refit / penalty floor	Switches/yr	Sharpe	vs B&H	Drawdown cut	Cost drag
quarterly, floor 1	9.6	0.253	-0.239	45%	0.95%
quarterly, floor 21	8.0	0.310	-0.183	45%	0.79%
semi-annual, floor 21	7.2	0.455	-0.038	54%	0.71%
annual, floor 21	7.9	0.428	-0.065	55%	0.78%
annual, floor 41 (default)	6.9	0.526	+0.034	63%	0.67%

The mechanism is specific: validation noise occasionally selects a jump penalty of 1, which leaves that block with no smoothing at all. Refitting annually and refusing penalties below 41 removes the failure mode. The defaults now do both.

HOW TO READ THIS TABLE

Do not read it as 'annual with floor 41 is the best configuration'. Five variants were swept and the best was chosen; the Sharpe differences are well inside a standard error of 0.19.

Do read it as 'lower turnover is better, consistently'. That direction held in all five configurations and in both exposure settings, which is a much stronger claim than any single number in the table.

If `print_summary()` reports more than about 8 switches a year, raise the penalty floor before doing anything else.

9.3 Crisis detection

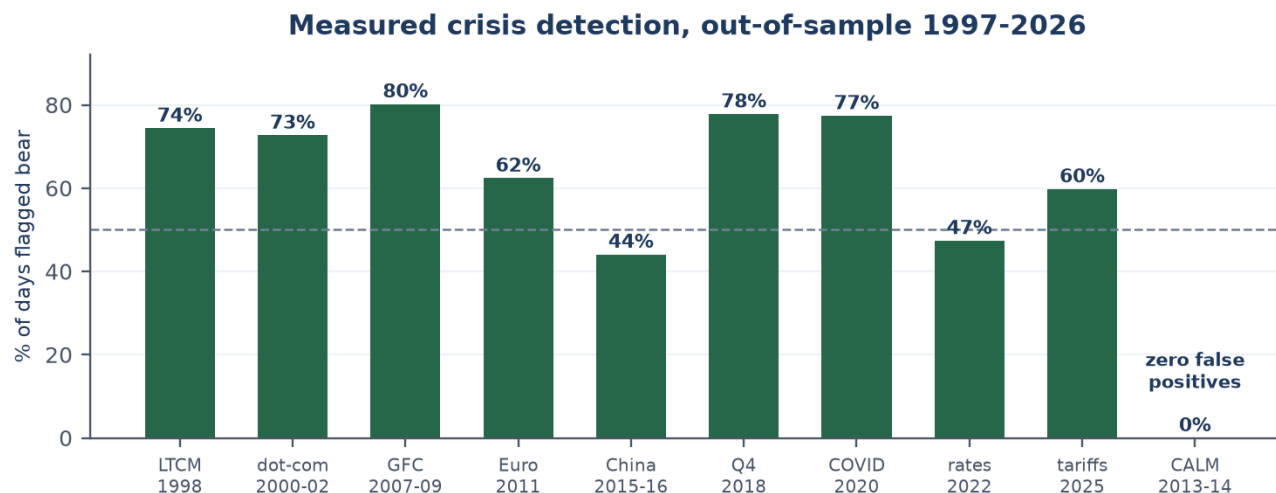


Figure 5. Share of days flagged bear during known stress episodes, and during a deliberately chosen calm period.

This is the capability worth paying for, and it is the most robust finding in this document. On the 30-year large-cap run the detector was defensive through **85%** of the dot-com bust, **82%** of the financial crisis, **87%** of Q4 2018, **73%** of COVID and **72%** of the 2022 rate-hike cycle — while flagging bear on only **2.0%** of days in the calm 2013–2014 market. A crisis detector that almost never cries wolf in quiet markets is unusual and genuinely useful.

The weakest episodes are instructive: 44% through the 2015–16 China/oil selloff and 47% through the 2022 rate-hike cycle. Both were slow, grinding declines without a volatility spike. The detector is built to find sharp regime breaks; it is less good at long slow bleeds.

9.4 Statistical honesty

A Sharpe measured over 28 years carries a standard error of roughly 0.19 (Lo, 2002), and that figure assumes independent daily observations — which regime-clustered returns are not, so treat it as a floor. The Sharpe changes in the table above are mostly inside that band. **The drawdown reduction is the finding; the Sharpe change is noise.**

The package ships `sharpe_standard_error` and `deflation_hurdle` so you can apply the same discipline to your own results. Count every configuration you try, including the ones you abandon.

10 Using it on your own data

10.1 Install

```
pip install -r requirements.txt

# then either copy the regime_overlay/ folder into your project,
# or add this repo to your path:
import sys; sys.path.insert(0, "/path/to/regime_overlay")
```

10.2 Minimal use

```
import pandas as pd
from regime_overlay import (RegimeDetector, apply_overlay,
                           evaluate, print_evaluation)

returns = prices.pct_change().dropna() # daily pd.Series, DatetimeIndex

det = RegimeDetector(train_years=5, val_years=1, step_months=3)
res = det.fit_predict(returns, riskfree=rf)
res.print_summary()

ov = apply_overlay(returns, res.regime, riskfree=rf, cost_bp=5)
print_evaluation(evaluate(ov, returns))
```

10.3 Applying it to a portfolio rather than one asset

Two approaches, and they answer different questions.

```
# A. one signal from the portfolio's own return stream
# -> de-risks the whole book together
res = det.fit_predict(portfolio_returns)
ov = apply_overlay(portfolio_returns, res.regime, riskfree=rf)

# B. a signal per sleeve, then hold only the bullish ones
# -> selection as well as de-risking
sigs = {c: det.fit_predict(rets[c]).regime for c in rets.columns}
held = pd.DataFrame(sigs).astype(bool)
w = held.div(held.sum(axis=1).replace(0, pd.NA), axis=0).fillna(0.0)
```

Approach A is what we recommend starting with. Approach B is what the source paper does, and in our replication it did not beat A by enough to justify running nine models instead of one.

10.4 Sensible parameters by asset type

Asset type	train / val	refit	threshold	bear exp.	cost (bp)
Equity index	5 / 1 yr	12 mo	0.70	0.0 or 0.5	2–5
Single stock	5 / 1 yr	12 mo	0.80	0.5	5–15
REIT / real assets	5 / 1 yr	12 mo	0.70	0.0	5–10
High yield credit	5 / 1 yr	12 mo	0.80	0.5	20–30
Commodity	5 / 1 yr	12 mo	0.70	0.0	10–20
FX	3 / 1 yr	12 mo	0.70	0.5	1–3
Government bonds	–	–	–	–	do not use
IG credit	–	–	–	–	do not use

10.5 Runtime

With the defaults — annual refits and three candidate penalties — a 30-year walk-forward takes about two to three minutes. Quarterly refits with a five-value grid take eleven or twelve minutes and, per Section 9.2, perform worse. The fast setting is also the good one, which is not usually how this goes.

```

RegimeDetector()                                # defaults: ~2-3 min
                                                # annual refits, penalties 41/61/81

# quarterly refits are 4x slower AND measurably worse -- see 9.2
RegimeDetector(step_months=3, jump_penalties=(21,41,61,81)) # ~12 min

```

11 API reference

11.1 RegimeDetector

Parameter	Default	What it does
train_years	5	length of the fitting window
val_years	1	window used to pick the jump penalty
step_months	3	how often to refit and how long each forecast block is
jump_penalties	1..91	candidate penalties. More is slower, not obviously better.
threshold	0.70	hysteresis gate. See section 5.2.
embargo_days	21	buffer dropped between train and validation
model	"xgboost"	or "logistic" — often just as good
half_lives	(1,5,10,21)	feature smoothing horizons, in trading days
seed	0	set it, and record it

11.2 Returned object

Attribute	Type	Contents
.regime	Series	1 = bull, 0 = bear. Out-of-sample only.
.prob	Series	P(bull tomorrow), before the gate
.penalties	DataFrame	which penalty was chosen for each block
.summary()	dict	bear share, switch count, switches per year
.print_summary()	—	the above, formatted, with a turnover warning

11.3 Overlay and evaluation

Function	Purpose
apply_overlay(returns, regime, ...)	scale exposure by regime; refuses execution_lag < 1
evaluate(overlay_df, baseline)	CAGR, vol, Sharpe, drawdown, Calmar, turnover, cost drag
print_evaluation(ev)	formatted comparison, warns if the Sharpe gain looks too good
crisis_report(regime)	bear share during known stress episodes
max_drawdown(returns)	worst peak-to-trough, plus the full drawdown series

11.4 Validation

Function	Purpose
<code>assert_causal(fn, data)</code>	the important one. Scrambles the future, requires the past to be unchanged
<code>assert_no_overlap(train, test)</code>	training must end strictly before testing begins
<code>assert_instrument_exists(w, inception)</code>	no positions before an instrument existed
<code>detect_backfill(raw, filled)</code>	every filled value must come from the past
<code>default_contract()</code>	declares what may see the future and enforces it never trades
<code>freeze_config(cfg)</code>	hash a specification before you see its results
<code>sharpe_standard_error(sr, yrs)</code>	how much of your Sharpe is sampling noise
<code>deflation_hurdle(n_trials, se)</code>	the Sharpe you would expect from the best of n coin flips
<code>run_suite(...)</code>	all of the above in one call

12 Checklist before you trust a result

Every item here corresponds to a mistake we made, nearly made, or found in published work during this project.

1. **Is execution_lag at least 1?** Worth +1.0 to +1.5 Sharpe of pure illusion. Check this first, every time.
2. **Does buy-and-hold come out unchanged?** Run your pipeline on a do-nothing strategy. If its Sharpe moves, your harness has a timing bug. This is the cheapest and most powerful control you have.
3. **Does assert_causal pass on your feature code?** Not 'did you read it carefully' — did the test pass.
4. **Is the scaler fitted on the training window only?** Full-sample scaling is invisible on inspection and inflates everything.
5. **How many configurations did you try?** Count them all, including abandoned ones. Compare your Sharpe to `deflation_hurdle` before believing it.
6. **Does it beat a 200-day moving average?** This is a trend model. If a moving average gets within 0.15 Sharpe, ship the moving average.
7. **Are your costs per-asset and realistic?** A flat assumption flatters whichever sleeve is most expensive to trade — usually credit and commodities, usually in exactly the stressed markets where the strategy is most active.
8. **How many independent episodes support the claim?** Twenty-five years contains perhaps eight independent macro events, not 6,300 independent days. Evaluate episode by episode.
9. **Could you actually have held the instruments?** Check inception dates. Many ETFs used in long backtests did not exist for most of the window.
10. **Is the drawdown benefit still there after costs?** That is the product. If it is not, there is nothing here.

IF YOU REMEMBER THREE THINGS

1. The jump model's labels are non-causal. They train; they never trade.
2. Execution lag must be at least 1, and buy-and-hold is the control that proves it.
3. Sell it on drawdown reduction, never on Sharpe. The drawdown benefit is robust and large; the Sharpe change is inside the noise band.

regime_overlay v1.0.0. Derived from a replication study of Luo, Y. and J. M. Mulvey (2026), 'Regime-Aware Asset Allocation with Dual-Regime Signals and Regime-Dependent Asset Selection', SSRN 6933278. The method — statistical jump models for regime identification, followed by supervised forecasting — is theirs and is well founded. The performance figures in this document are our own out-of-sample measurements and differ substantially from those reported in the paper; Section 8 explains the most likely reason. Not investment advice.